

Airflow Data Pipeline – Data Quality & Task Monitoring System

Document Type: Project Requirement + Technical Design Document

Version: 1.0

Status: Development Ready

Technology: Python + Apache Airflow

Execution: Local → Production/Cloud

1. Executive Summary

The **Airflow Data Pipeline – Data Quality & Task Monitoring System** is a production-oriented data engineering project designed to process data through multiple pipeline stages while continuously monitoring:

- Task execution status
- Task completion percentage
- Source record count
- Input/output record count at every stage
- Data loss or unexpected record differences
- Data-quality checks
- Pipeline execution time
- Errors and failures
- Final source-to-target reconciliation

The system will initially run locally using Python and Apache Airflow. The architecture will be designed so that the same solution can later be deployed to AWS, GCP, or another cloud platform.

The key objective is:

Don't just check whether the Airflow task succeeded; also verify whether the expected data successfully moved through every stage.

2. Business Problem

In a traditional ETL pipeline, Airflow may show:

None

Extract	SUCCESS
Transform	SUCCESS
Load	SUCCESS

However, this does not necessarily mean that the data is correct.

For example:

None

Source	100,000 records
↓	
Extract	100,000
↓	
Validation	99,800
↓	
Transformation	99,800
↓	
Filter	95,000
↓	
Target	94,500

Airflow may still show all tasks as successful.

The business needs to know:

- Where did records decrease?
- Was the decrease expected?
- How many records were rejected?
- Was data lost unexpectedly?
- Did the target receive all expected records?
- Which task caused the problem?
- What percentage of the pipeline is complete?

3. Project Objectives

1. Build an Airflow-based ETL pipeline.
 2. Execute multiple independent pipeline tasks.
 3. Capture source record count.
 4. Capture input/output count at every stage.
 5. Compare counts between stages.
 6. Calculate record difference.
 7. Calculate record difference percentage.
 8. Calculate task completion percentage.
 9. Calculate data processing percentage.
 10. Perform data-quality checks.
 11. Detect unexpected data loss.
 12. Detect duplicate records.
 13. Detect null values in mandatory columns.
 14. Validate schema and data types.
 15. Generate pipeline execution reports.
 16. Maintain an audit trail.
 17. Support retries and failure handling.
 18. Run the same processing logic directly through Python.
 19. Run the complete workflow through Airflow.
 20. Keep the architecture ready for cloud deployment.
-

4. Scope

4.1 In Scope

- Python ETL
- Apache Airflow
- Local data source
- Local target
- Row-count monitoring
- Data-quality framework
- Task monitoring
- Error handling
- Local execution
- Production-oriented architecture

4.2 Out of Scope

- Real-time streaming
 - Full enterprise BI dashboard
 - Automatic correction of bad data
 - Complex machine-learning models
 - Production cloud deployment
-

5. Proposed Technology Stack

Layer	Technology
Programming	Python
Orchestration	Apache Airflow
Local Database	PostgreSQL / SQLite
Data Processing	Pandas / SQL
Source	CSV / Parquet / Database
Target	PostgreSQL initially
Testing	Pytest
Configuration	YAML / Environment Variables
Version Control	Git
Containerization	Docker
Production	AWS / GCP